CampusLoop Security Audit & Tiered Remediation Guide

Full Static Repository Review, Code Evidence, PR #1 Fixes & Next-Phase Engineering Suggestions

Repositories: 2labor/CampusLoop-Backend- (branch: security-audit, PR #1) & 2labor/CampusLoop-Mobile- | Date: 2 November 2026

1. Executive Summary & Security Tier Definitions

This document provides the complete security audit and remediation blueprint for the **CampusLoop** platform (Spring Boot 3.2.5 / Kotlin 1.9.23 backend and Android Jetpack Compose mobile client). It evaluates the codebase against OWASP API Security Top 10 (2023) and the CampusLoop privacy-by-design product requirements, documents the code fixes shipped in **Pull Request #1 (`security-audit` branch)**, and prescribes concrete next-step engineering suggestions grouped by security tier.

Security Tier	Release Gate	Risk Profile & Engineering Requirement
TIER 0 • P0 CRITICAL	**Immediate / Pre-Integration**	Direct unauthorized data access (BOLA/IDOR), cross-user realtime topic eavesdropping, or exposed database/Redis infrastructure. Must be fixed before any shared host or mobile integration.
TIER 1 • P1 HIGH	**Before MVP Release**	Authentication bypasses, domain spoofing, account enumeration, capacity race conditions, schema constraint defects, and exact location leakage without privacy projections.
TIER 2 • P2 MEDIUM	**Before Production Launch**	Database migration governance (Flyway), TLS & HTTP security headers, container hardening, dependency vulnerability scanning, and automated security regression suites.
TIER 3 • P3 LOW	**Defense-in-Depth / Post-MVP**	Mobile client Keystore encryption, certificate pinning, structured security audit trails, PII log redaction, and automated abuse/moderation reporting workflows.

2. Master Findings & Remediation Matrix

ID	Tier	OWASP Category	Vulnerability Summary	PR #1 Status	Next Action
SEC-01	TIER 0 (P0)	API8: Security Misconfiguration	Hardcoded DB credentials (`user/password`), unauthenticated Redis, public `0.0.0.0` ports, unpinned `:latest` images.	FIXED IN PR #1	Use Secret Manager & TLS in cloud
SEC-02	TIER 0 (P0)	API1: Broken Object Level Auth (BOLA)	Missing caller checks on private groups, closed event participants, self-accepted friend requests, blocked profiles, and `SecurityConfig` path matcher.	FIXED IN PR #1	Add invite table for closed events
SEC-03	TIER 0 (P0)	API1 / WebSocket Authz	Wildcard WebSocket origins (`*`) and missing STOMP `SUBSCRIBE` authorization on `/topic/users/{id}/**` .	FIXED IN PR #1	Authorize event/group WS topics
SEC-04	TIER 1 (P1)	API2: Broken Authentication	Verification check commented out; `email.endsWith()` allowed lookalike suffix domains (e.g., `evil-tudublin.ie`).	FIXED IN PR #1	Store immutable `verifiedAt` timestamp
SEC-05	TIER 1 (P1)	API4: Unrestricted Resource Consumption	Password reset & verification threw distinct user-existence errors, used non-crypto `Random` , and lacked rate limits.	FIXED RNG/ENUM	Add Redis Bucket4j rate limiter
SEC-06	TIER 1 (P1)	API3 / Input Validation	Missing `@Valid` on controllers, unbounded text/coordinates/capacity on DTOs, and uncapped Spring `Pageable` size.	FIXED IN PR #1	Cap max page size in `application.yml`
SEC-07	TIER 1 (P1)	API2: Broken Authentication	Weak JWT secret handling, no `iss` / `jti` claims, 24h static token with no refresh rotation or password-reset revocation.	FIXED KEY/CLAIMS	Add refresh tokens & Redis revocation
SEC-08	TIER 1 (P1)	API6: Business Flow / Race Condition	`EventParticipant` unique constraint used wrong column (`account_id` vs `user_id`); concurrent joins bypassed `maxParticipants` .	FIXED IN PR #1	Add PostgreSQL concurrency test
SEC-09	TIER 1 (P1)	API5: Broken Function Level Auth	`GroupMember` lacked unique constraint; `getGroupMembers` queried by `operatorId` as PK; swapped args in `GroupMembersController` .	FIXED IN PR #1	Add admin ownership transfer endpoint
SEC-10	TIER 1 (P1)	Realtime Integrity	Redis channel mismatch (`ws:sendToUser:` vs `ws:user:*`) and malformed colon prefix stripping on broadcast.	FIXED IN PR #1	Add transactional outbox & cursor sync
SEC-11	TIER 1 (P1)	API3: Excessive Data Exposure	Exact event coordinates and full profile fields returned to all callers without privacy projections or location sharing modes.	PLANNED (MVP)	Implement `HIDDEN / FRIENDS / EVENT`
SEC-12	TIER 2 (P2)	API8: Security Misconfiguration	Missing `application.yml` , Flyway migrations, global `@RestControllerAdvice` error envelope, and dependency scanning.	FIXED WRAPPER	Add Flyway, GlobalExceptionHandler, CI
SEC-13	TIER 2 (P2)	Security Verification	Only empty context-load test existed; no unit or integration tests for BOLA, JWT, or domain boundaries.	5 UNIT TESTS ADDED	Add MockMvc & Testcontainers suite

Tier 0 (P0 Critical) — Immediate Security Blockers

Must be enforced before exposing the backend to any shared network or connecting the Android client

SEC-01: Infrastructure Exposure, Default Credentials & Unauthenticated Redis

REMEDIATED IN PR #1

Vulnerability Evidence: `docker-compose.yml` previously ran `postgres:latest` with hardcoded `POSTGRES_USER: user` / `POSTGRES_PASSWORD: password` published on `0.0.0.0:5435`, and `redis:latest` without authentication on `0.0.0.0:6379`.

Fix Applied in `security-audit` (PR #1):

- Pinned images to `postgres:16-alpine` and `redis:7.2-alpine` on an isolated bridge network (`campusloop_internal`).
- Replaced hardcoded secrets with mandatory environment variable checks (`${POSTGRES_PASSWORD:?...}`, `${REDIS_PASSWORD:?...}`) and added `.env.example`.
- Bound host ports strictly to loopback (`127.0.0.1:${POSTGRES_PORT:-5435}:5432` and `127.0.0.1:${REDIS_PORT:-6379}:6379`) and enabled `--requirepass` on Redis.

Further Suggestions to Fix (Cloud / Production Deployment):

- Place PostgreSQL and Redis in private subnets with no public IP; allow ingress only from the Spring Boot application security group.
- Enable TLS in transit for both PostgreSQL (`sslmode=verify-full`) and Redis (`rediss://`), and inject credentials via AWS Secrets Manager, GCP Secret Manager, or GitHub Actions encrypted secrets.

SEC-02: Broken Object-Level Authorization (BOLA / IDOR) Across Core Resources

REMEDIATED IN PR #1 + PHASE 2 EXTENSION

Vulnerability Evidence:

1. `SecurityConfig.kt:29` used `.requestMatchers("api/v**/auth/**").permitAll()` (missing leading slash and overly broad wildcard), which left `POST /api/v1/auth/password/change-password` unauthenticated at the filter chain level.
2. `FriendshipServiceImpl.kt:208` used an undirected lookup (`findBetweenUsers`) inside `findPendingRequest` without checking `request.target.id == currentId`, allowing a user to send a friend request and immediately accept their own request on behalf of the victim.
3. `GroupController.kt:48` and `GroupServiceImpl.kt:56` returned private groups by ID without checking caller membership, and `getGroups` / `searchGroups` returned private groups to all users.
4. `EventParticipantController.kt:29` allowed any user to enumerate participants of `CLOSED` events without being the organizer or an approved participant.
5. `UserProfileController.kt:34` returned user profiles without checking whether a `BLOCKED` relationship existed between caller and target.

Fix Applied in `security-audit` (PR #1):

- Restricted `SecurityConfig` `permitAll()` strictly to `/api/v1/auth/register`, `/api/v1/auth/login`, `/api/v1/auth/verification/**`, `/api/v1/auth/password/forgot`, and `/api/v1/auth/password/reset`.
- Updated `FriendshipServiceImpl.findPendingRequest` to query `repository.findByInitializerAndTarget(requesterUser, currentUser)` and throw `AccessDeniedException` if the caller is not the target recipient.
- Enforced active membership checks in `GroupServiceImpl.getGroupById(operatorId, id)` and filtered `GroupRepository` public list/search queries with `isPrivate = false`.
- Enforced caller authorization in `EventParticipantServiceImpl.getEventParticipants` for `EventType.CLOSED` events and block-status filtering in `UserProfileServiceImpl.getProfileForViewer`.

Further Suggestions to Fix (Closed Event Read Visibility & Global Block Filtering):

```
// Suggested addition to EventServiceImpl.getEventDetails(callerId: UUID, eventId: UUID):
val event = repository.findById(eventId).orElseThrow { ResourceNotFoundException("Event not found") }
if (event.eventType == EventType.CLOSED && event.creator.id != callerId) {
    val isParticipant = participantRepository.existsByEventIdAndUserId(eventId, callerId)
    if (!isParticipant) throw AccessDeniedException("Closed event details require an invitation or participation")
}
```

- Exclude users who have blocked (or been blocked by) the caller from `GET /api/v1/users/search` and `GET /api/v1/events/upcoming` queries at the JPA repository level.

SEC-03: Wildcard WebSocket Origin & Unrestricted STOMP Topic Subscriptions

REMEDIATED IN PR #1 + TOPIC SCOPE EXTENSION

Vulnerability Evidence: `WebSocketConfig.kt:25` configured `.setAllowedOriginPatterns("*")`, and `JwtChannelInterceptor.kt:21` only inspected `StompCommand.CONNECT`. Any authenticated client could send a `SUBSCRIBE` frame to `/topic/users/{victimUuid}/friends` and intercept realtime friendship/block events for other students.

Fix Applied in `security-audit` (PR #1):

- Externalized allowed origins in `WebSocketConfig` via `@Value("\${campusloop.websocket.allowed-origins:http://localhost:*,https://localhost:*}")`.
- Added `StompCommand.SUBSCRIBE` interception in `JwtChannelInterceptor` that verifies the authenticated principal's UUID matches the target user segment in `/topic/users/{userId}/**` and `/user/{userId}/**`.

Further Suggestions to Fix (Resource-Scoped Topic Authorization):

- In `JwtChannelInterceptor.authorizeSubscription`, also intercept `/topic/events/{eventId}/participants` and verify via `EventParticipantRepository` that the subscribing user is permitted to view that event before allowing the STOMP subscription.

Tier 1 (P1 High) — Authentication, Rate Limiting & Session Security

Required before MVP release to protect university onboarding, credentials, and token lifecycle

SEC-04: Account Verification Enforcement & University Domain Boundary Check

REMEDIATED IN PR #1

Vulnerability Evidence: `AuthFacade.kt:33` validated campus emails using `!command.email.endsWith(universityDomain, ignoreCase = true)`, which could be bypassed by registering `attacker@evil-tudublin.ie` when `universityDomain = "tudublin.ie"`. In addition, lines 48 and 61–63 commented out email verification sending and login verification enforcement.

Fix Applied in `security-audit` (PR #1):

- Implemented `AuthFacade.isAllowedUniversityEmail(email, configuredDomain)` which extracts the domain after `@` and requires either an exact match (`emailDomain == allowedDomain`) or a dot-separated subdomain match (`emailDomain.endsWith(".$allowedDomain")`).
- Normalized registration email and login to lowercase and re-enabled verification code dispatch and login verification checks (controlled by `campusloop.auth.require-verification: true`).

SEC-05: Account Enumeration, Weak OTP RNG & Missing Auth Rate Limiting

RNG & ENUMERATION FIXED IN PR #1 • RATE LIMITER SUGGESTED

Vulnerability Evidence: `AccountPasswordServiceImpl.kt:28-31` and `AccountVerificationServiceImpl.kt:23-30` threw `IllegalArgumentException("Account with email ... not found")` when an unknown email was submitted, allowing attackers to enumerate registered students. Both services generated 6-digit OTPs using non-cryptographic `kotlin.random.Random.nextInt(100000, 999999)` with no attempt counter or rate limit.

Fix Applied in `security-audit` (PR #1):

- Replaced `kotlin.random.Random` with `java.security.SecureRandom` (`100000 + secureRandom.nextInt(900000)`).
- Updated `forgotPassword` and `sendVerificationCode` to return silently when an email is not registered or already verified, and standardized login failure messages to `"Invalid credentials"`.

Further Suggestions to Fix (Redis Sliding-Window Rate Limiter & Max OTP Attempts):

```
// Suggested Redis Rate Limiter Service to inject into AuthController, PasswordController & VerificationController:
@Service
class RedisRateLimiter(private val redisTemplate: StringRedisTemplate) {
    fun checkLimit(key: String, maxRequests: Long, window: Duration) {
        val redisKey = "ratelimit:$key"
        val count = redisTemplate.opsForValue().increment(redisKey) ?: 1L
        if (count == 1L) redisTemplate.expire(redisKey, window)
        if (count > maxRequests) throw TooManyRequestsException("Rate limit exceeded. Please try again later.")
    }
}
```

- Enforce: **5 login attempts / 5 min** per IP + username; **3 OTP send requests / 15 min** per email; and **5 OTP verification attempts** before invalidating the active OTP token in `RedisTokenServiceImpl`.

SEC-07: JWT Secret Validation, Claims & Refresh Token Revocation Lifecycle

KEY & CLAIMS FIXED IN PR #1 • REFRESH ROTATION SUGGESTED

Vulnerability Evidence: `JwtCore.kt` did not validate secret key byte length on startup, omitted `jti` and `iss` claims, and issued 24-hour stateless access tokens that remained valid even after a user reset or changed their password.

Fix Applied in `security-audit` (PR #1):

- Added fail-fast startup check in `JwtCore.init` requiring at least 32 UTF-8 bytes (256 bits) for the HMAC-SHA256 signing key.
- Added unique `jti` (`UUID.randomUUID()`) and `iss` (`campusloop-api`) claims and enforced `.requireIssuer(issuer)` during token parsing.

Further Suggestions to Fix (Short-Lived Access JWT + Redis Refresh Token Rotation):

- Reduce access token TTL to **15 minutes** (`900000` ms) and issue an opaque 256-bit refresh token stored hashed in Redis under `session:refresh:{userId}:{sessionId}` with a 14-day TTL.
- Store a `security:pwd_changed_at:{userId}` timestamp in Redis/Account whenever `resetPassword` or `changePassword` runs, and reject any JWT whose `iat` is older than `pwd_changed_at`.

Tier 1 (P1 High) — Validation, Concurrency, Integrity & Location Privacy

Protecting database invariants, concurrent event capacity, group roles, and student location privacy

SEC-06: Input Validation, Coordinate Bounds & Pagination Limits

REMEDIATED IN PR #1 + PAGEABLE CAP CONFIG

Vulnerability Evidence: `AuthController`, `GroupController`, and `UserProfileController` omitted `@Valid` on `@RequestBody` parameters, and `CreateEventRequest` / `UpdateEventRequest` / `CreateGroupRequest` / `UpdateProfileRequest` had no field constraints, allowing blank titles, negative capacity, out-of-range coordinates (e.g., latitude `999.0`), or megabyte-sized strings.

Fix Applied in `security-audit` (PR #1):

- Added `@Valid` to all mutating controller endpoints across `AuthController`, `GroupController`, and `UserProfileController`.
- Added `@NotBlank`, `@Size`, `@Future`, `@DecimalMin("-90.0")`/`@DecimalMax("90.0")`, `@DecimalMin("-180.0")`/`@DecimalMax("180.0")`, and `@Min(2)`/`@Max(500)` annotations across all request DTOs.

Further Suggestions to Fix (Global Pageable Cap):

- Configure `spring.data.web.pageable.max-page-size=50` and `spring.servlet.multipart.max-request-size=2MB` in `application.yml` so clients cannot request `?size=1000000` on slice endpoints.

SEC-08 & SEC-09: Schema Constraints, Capacity Locking & Group Membership Defects

REMEDIATED IN PR #1

Vulnerability Evidence:

- **SEC-08:** `EventParticipant.kt:23` declared `UniqueConstraint(columnNames = ["event_id", "account_id"])` while the join column at line 35 was `user_id`, breaking schema generation. Furthermore, `EventParticipantServiceImpl.joinToEvent` checked `countByEventIdAndStatus` without locking the `Event` row, allowing concurrent requests to exceed `maxParticipants`.
- **SEC-09:** `GroupMember.kt` had no unique constraint on `(group_id, user_id)`. `GroupMemberServiceImpl.getGroupMembers` called `repository.findByIdOrNull(operatorId)` (passing a `User` UUID as a `GroupMember` primary key). `GroupMemberServiceImpl.addMember` looked up `operatorId` instead of `targetId` without checking moderator/admin role. Finally, `GroupMembersController.joinGroup` and `leaveGroup` passed `(groupId, userId)` in reversed order!

Fix Applied in `security-audit` (PR #1):

- Fixed `EventParticipant` unique constraint to `["event_id", "user_id"]` and added `@Lock(LockModeType.PESSIMISTIC_WRITE)` (`EventRepository.findByIdWithLock`) in `joinToEvent` and `updateParticipant`.
- Added `UniqueConstraint(columnNames = ["group_id", "user_id"])` to `GroupMember`, fixed `getGroupMembers` and `addMember` in `GroupMemberServiceImpl`, protected the group creator from demotion/banning, and fixed the swapped arguments in `GroupMembersController`.

SEC-10 & SEC-11: Realtime Channel Alignment & Location Privacy Projections

SEC-10 FIXED IN PR #1 • SEC-11 DESIGN READY

Fix Applied for SEC-10 in PR #1: Aligned `RedisWebSocketBroadcaster.sendToUser` to publish to `ws:user:{userId}:{destination}` matching `RedisConfig`'s `PatternTopic("ws:user:*")`, and fixed `RedisWebSocketReceiver.handleBroadcast` to strip `"ws:broadcast:"` including the colon.

Suggestions to Fix SEC-11 (Location Privacy by Design Before Map Integration):

- **Never expose raw user GPS coordinates by default:** Implement a `LocationShare` entity with `mode` enum (`HIDDEN` default, `FRIENDS`, `DURING_EVENT`) and mandatory `expiresAt` TTL.
- **Coarse Coordinate Rounding for Feeds:** For non-approved viewers or broad map discovery, round event/friend coordinates to 3 decimal places (~110m campus zone resolution) or return only the named campus building (`locationName`) until the user is an approved participant or accepted friend.

Tier 2 (P2 Medium) & Tier 3 (P3 Low) — Hardening & Mobile Security

Pre-production operational resilience, standardized error envelopes, CI security gates, and Android client security

SEC-12: Standardized Error Handling, Flyway Migrations & HTTP Security Headers

MAVEN WRAPPER FIXED IN PR #1 • CONFIG BLUEPRINT BELOW

Problem: Unhandled `IllegalArgumentException`, `IllegalStateException`, and `AccessDeniedException` currently rely on Spring's default error page/response, which can leak internal exception details or return inconsistent HTTP status codes (e.g., `500` instead of `400` / `403` / `409`).

Suggestion to Fix — Add a Global `@RestControllerAdvice` Error Envelope:

```
@RestControllerAdvice
class GlobalExceptionHandler {
    @ExceptionHandler(AccessDeniedException::class)
    fun handleAccessDenied(ex: AccessDeniedException) =
        ResponseEntity.status(HttpStatus.FORBIDDEN).body(MessageResponse("Access denied"))

    @ExceptionHandler(IllegalArgumentException::class)
    fun handleBadRequest(ex: IllegalArgumentException) =
        ResponseEntity.status(HttpStatus.BAD_REQUEST).body(MessageResponse(ex.message ?: "Invalid request"))

    @ExceptionHandler(IllegalStateException::class)
    fun handleConflict(ex: IllegalStateException) =
        ResponseEntity.status(HttpStatus.CONFLICT).body(MessageResponse(ex.message ?: "Operation conflict"))

    @ExceptionHandler(MethodArgumentNotValidException::class)
    fun handleValidation(ex: MethodArgumentNotValidException): ResponseEntity<Map<String, Any>> {
        val errors = ex.bindingResult.fieldErrors.associate { it.field to (it.defaultMessage ?: "Invalid") }
        return ResponseEntity.badRequest().body(mapOf("code" to "VALIDATION_ERROR", "fieldErrors" to errors))
    }
}
```

- **Database Migrations:** Add `flyway-core` and `flyway-database-postgresql` to `pom.xml`, set `spring.jpa.hibernate.ddl-auto=validate`, and commit `V1__init_schema.sql` under `src/main/resources/db/migration/`.
- **Supply-Chain & CI Security:** Enable GitHub Dependabot for Maven & Docker, and run `./mvnw test` automatically on every Pull Request via GitHub Actions.

SEC-13: Security Regression & Integration Testing Suite

5 UNIT TESTS ADDED IN PR #1 • EXPAND WITH MOCKMVC

Implemented in PR #1: Added `SecurityAuditRemediationTests.kt` verifying: (1) strict university email domain boundary checks against lookalike domains, (2) `JwtCore` 256-bit key length & issuer enforcement, (3) `JwtChannelInterceptor` cross-user STOMP subscription blocking, (4) friend-request target-only acceptance authorization, and (5) private group member list access denial for non-members.

Further Suggestions to Fix:

- Add `@WebMvcTest` / `MockMvc` controller security tests verifying that unauthenticated requests to `/api/v1/auth/password/change-password`, `/api/v1/events/**`, and `/api/v1/groups/**` return `401/403`.
- Add a Testcontainers PostgreSQL test verifying that two concurrent threads calling `joinToEvent` on an event with `maxParticipants = 2` cannot exceed capacity.

TIER 3 (P3 Low / Defense-in-Depth): Android Client (CampusMates) & Audit Logging

MOBILE INTEGRATION CHECKLIST

- **Secure Token Storage on Android:** Store the access/refresh JWT in Jetpack DataStore encrypted with `AndroidKeyStore` (`AES256_GCM`); never log tokens in OkHttp `HttpLoggingInterceptor` in release builds.
- **Network Security Configuration:** Add `res/xml/network_security_config.xml` with `cleartextTrafficPermitted="false"` for all non-debug domains so production traffic strictly enforces HTTPS/TLS 1.3.
- **Navigation by ID Only:** Keep passing only `eventId`, `userId`, and `groupId` in Compose Navigation routes (as implemented in Milestone 1) and re-fetch authorized projections from the repository rather than passing serialized sensitive payloads in navigation arguments.
- **Privacy-Safe Audit Trail:** Log security-sensitive events (failed logins, password resets, account blocks, role changes) with `userId`, `action`, and `requestId`, while redacting emails, passwords, OTP codes, and GPS coordinates from application logs.

Verification Summary: All Tier 0 (P0) and core Tier 1 (P1) code remediations are committed to branch `security-audit` (commit `0b66aeb`), verified with Maven 3.10.0 on JDK 21 (`SecurityAuditRemediationTests`: 5/5 passing), and submitted in Pull Request #1.